

Práctica 1 - Redes y Acceso Remoto

Objetivo

Explorar y configurar distintos esquemas de acceso remoto:

- Control compartido (host y cliente ven lo mismo).
- Acceso independiente (máquina virtual como equipo separado).
- Acceso directo a máquina virtual (cliente se conecta sin depender del host).

Fase 1 - Escritorio remoto directo (control compartido)

El usuario remoto comparte el mismo escritorio que el usuario local. Todas las acciones realizadas con mouse y teclado se reflejan en ambos lados.

Herramientas sugeridas

- TeamViewer, AnyDesk, Chrome Remote Desktop.

Tareas

- Instalar la herramienta en dos equipos (host y cliente).
- Configurar el acceso remoto (ID, contraseña o cuenta).
- Conectarse desde el cliente al host.
- Verificar el control compartido.
- Evaluar latencia y calidad de conexión.

Puntaje: 30%

Fase 2 - Máquina virtual con acceso físico independiente

La máquina virtual funciona como un equipo separado, con su propio escritorio y control independiente. Se simula un escenario tipo thin client o multiestación.

Herramientas sugeridas

- VirtualBox, VMware Workstation.

Tareas

- Crear una máquina virtual con sistema operativo.
- Configurar acceso remoto (RDP o VNC).
- Simular uso independiente.
- Verificar que host y VM operen simultáneamente.
- Documentar limitaciones de hardware.

Puntaje: 30%

Fase 3 - Acceso remoto a escritorio de máquina virtual

El cliente accede directamente al escritorio de la máquina virtual, sin interacción visible en el host. Se asemeja a la conexión con un servidor remoto.

Herramientas sugeridas

- Remote Desktop Protocol (RDP), VNC.

Tareas

- Configurar la IP de la máquina virtual (modo bridge o NAT con redirección).
- Habilitar acceso remoto dentro de la VM.
- Conectarse desde otro equipo directamente a la VM.
- Comparar con el acceso remoto directo de la fase 1.

Puntaje: 40%

Entrega Final

El informe debe presentarse de manera formal y estructurada, incluyendo los siguientes elementos:

Documentación paso a paso

Cada fase debe estar explicada con capturas de pantalla y notas breves que describan lo realizado. Las notas deben señalar configuraciones relevantes, dificultades encontradas y soluciones aplicadas.

Diagramas de arquitectura

Incorporar diagramas simples que representen la relación entre host, cliente y máquina virtual. Los diagramas deben mostrar claramente el flujo de conexión y los roles de cada equipo.

Comparaciones técnicas

Elaborar una tabla comparativa entre los distintos esquemas de acceso remoto, destacando diferencias en latencia, independencia de uso, facilidad de configuración y limitaciones de hardware.

Análisis final y conclusiones

Redactar un análisis integrador que compare los tres escenarios. Identificar ventajas y desventajas de cada esquema y relacionar los resultados con situaciones reales, tales como soporte remoto en empresas, uso de aulas virtuales y laboratorios académicos, o administración de servidores empresariales. Proponer casos de uso donde cada esquema resulte más adecuado.

Práctica 2 - Desarrollo remoto con VS Code y servidor Linux

Configurar un entorno en el cual, desde un sistema Windows, se pueda establecer conexión con un servidor Ubuntu para desarrollar y ejecutar código de forma remota, simulando un entorno cliente-servidor real. Al finalizar la práctica, el estudiante comprenderá los fundamentos del desarrollo remoto y podrá evaluar sus ventajas frente al desarrollo local.

Fase 1 - Preparación del servidor Ubuntu

Instalar y preparar un servidor Linux accesible por red.

Herramientas

- Ubuntu Server, OpenSSH.

Tareas

- Instalar Ubuntu Server en una máquina física o virtual.
- Configurar la red con una dirección IP fija o un DHCP reconocible.
- Instalar y habilitar el servicio SSH.
- Verificar la conexión local mediante acceso remoto al servidor.

Puntaje: 20%

Fase 2 - Conexión desde Windows con VS Code

Configurar Visual Studio Code para trabajar directamente sobre el servidor Ubuntu.

Herramientas

- Visual Studio Code, extensión Remote - SSH.

Tareas

- Instalar Visual Studio Code en Windows.
- Instalar la extensión Remote - SSH.
- Configurar el archivo de conexión SSH con los parámetros del servidor.
- Conectarse al servidor desde Visual Studio Code.
- Abrir una carpeta remota para trabajar directamente sobre ella.

Puntaje: 25%

Fase 3 — Ejecución de código remoto

Ejecutar programas directamente en el servidor desde Visual Studio Code.

Lenguajes sugeridos

- Python, Java, C/C++, Node.js.

Tareas

- Instalar los lenguajes de programación necesarios en Ubuntu.
- Crear un programa sencillo de prueba.
- Ejecutar el programa desde la terminal integrada de Visual Studio Code.
- Verificar que la ejecución ocurre en el servidor y no en el sistema local.

Puntaje: 25%

Fase 4 — Configuración de entorno servidor (servicios)

Simular un entorno real de servidor en el cual se ejecutan aplicaciones.

Opcional pero recomendado

- Servidor web, API básica.

Herramientas

- Apache HTTP Server, Node.js.

Tareas

- Instalar un servidor web (Apache o Node.js).
- Crear una página o API sencilla.
- Acceder desde el navegador del cliente en Windows.
- Verificar la comunicación cliente-servidor.

Puntaje: 15%

Fase 5 — Seguridad y análisis

Evaluar aspectos básicos de seguridad en el acceso remoto.

Tareas

- Cambiar el puerto de acceso SSH (opcional).
- Deshabilitar el inicio de sesión como usuario root.
- Analizar ventajas del desarrollo remoto.
- Comparar con el desarrollo local.

Puntaje: 15%

Entrega Final

El informe debe presentarse de manera formal y estructurada, incluyendo los siguientes elementos:

Documentación paso a paso

Cada fase debe estar explicada con capturas de pantalla y notas breves que describan lo realizado. Las notas deben señalar configuraciones relevantes, dificultades encontradas y soluciones aplicadas.

Diagramas de arquitectura

Incorporar diagramas simples que representen la relación entre cliente (Windows), servidor Ubuntu y las herramientas utilizadas. Los diagramas deben mostrar claramente el flujo de conexión y los roles de cada componente.

Código y evidencias

Incluir fragmentos de código fuente y capturas de ejecución remota. En el caso de servicios, mostrar la URL o IP funcional y evidencia de acceso desde el cliente.

Comparaciones técnicas

Elaborar una tabla comparativa entre desarrollo remoto y desarrollo local, destacando aspectos como rendimiento, facilidad de configuración, seguridad y escalabilidad.

Análisis final y conclusiones

Redactar un análisis integrador que evalúe las ventajas y desventajas del desarrollo remoto. Relacionar los resultados con situaciones reales, tales como administración de servidores, trabajo colaborativo en equipos distribuidos y entornos de producción. Proponer casos de uso donde el desarrollo remoto resulte más adecuado.

Práctica 3 - Orquestación de Contenedores

Objetivo

Implementar un proceso progresivo de orquestación con Docker, iniciando desde un entorno básico y avanzando hacia un clúster con Kubernetes, incluyendo una aproximación conceptual a la federación de clústeres. El estudiante deberá desplegar múltiples servicios distribuidos en diferentes nodos, alcanzando una arquitectura de hasta 12 máquinas con aplicaciones y bases de datos diversas, simulando un entorno empresarial real.

Fase 1 - Orquestación básica con Docker Swarm

Construir un clúster inicial utilizando Docker Swarm para comprender los fundamentos de la orquestación.

Herramienta

- Docker Swarm.

Tareas

- Inicializar el clúster Swarm.
- Configurar al menos dos nodos (manager y worker).
- Desplegar un servicio replicado (ejemplo: servidor web).
- Escalar el servicio a múltiples réplicas.
- Verificar el balanceo de carga entre nodos.

Puntaje: 20%

Fase 2 - Orquestación con Kubernetes (12 máquinas, múltiples servicios)

Desplegar un clúster con Kubernetes que gestione una arquitectura distribuida de 12 nodos, cada uno con servicios específicos. Se busca simular un entorno empresarial con aplicaciones web y bases de datos heterogéneas.

Herramienta

- Kubernetes

Arquitectura mínima

- 1 nodo maestro encargado de la administración del clúster.
- 2 nodos de trabajo principales que gestionan los pods y servicios.
- 12 máquinas virtuales o físicas distribuidas en los nodos, cada una ejecutando contenedores con aplicaciones o bases de datos

Servidores web

- Apache, Tomcat (Java), PHP.

Frameworks

- JSP, Struts.

Aplicaciones

- Python (Flask/Django).

Bases de datos

- SQL Server, PostgreSQL, MariaDB.

Tareas

- Instalar Kubernetes (Minikube, kubectl o k3s).
- Crear despliegues para cada servicio en nodos específicos.
- Exponer servicios mediante NodePort o LoadBalancer.
- Verificar comunicación entre pods y servicios distribuidos.
- Escalar al menos un servicio crítico (ejemplo: base de datos o servidor web).

Puntaje: 45%

Fase 3 - Integración multi-servicio (arquitectura distribuida)

Configurar una arquitectura en la que los servicios interactúen entre sí, simulando un flujo empresarial completo.

Tareas

- Configurar comunicación interna entre servicios mediante DNS de Kubernetes.
- Simular un flujo de trabajo: frontend, backend, base de datos y API.
- Probar tolerancia a fallos y recuperación automática de pods.

Puntaje: 20%

Fase 4 — Introducción a configuración federada

Explorar el concepto de federación de clústeres en Kubernetes como estrategia de alta disponibilidad y geodistribución.

Concepto clave

- Kubernetes Federation.

Tareas

- Investigar el funcionamiento de la federación en Kubernetes.
- Simular, de manera teórica o práctica, la existencia de dos clústeres independientes con despliegues replicados.
- Explicar las ventajas de la federación: redundancia, balanceo global, continuidad de servicio en múltiples regiones.

Documentos

- Diagrama de arquitectura federada.
- Explicación conceptual clara y detallada.

Puntaje: 15%

Entrega Final

El informe debe presentarse de manera formal y estructurada, incluyendo:

Documentación paso a paso

Cada fase debe estar acompañada de capturas de pantalla y notas explicativas que describan las configuraciones realizadas, dificultades encontradas y soluciones aplicadas.

Diagramas de arquitectura

Representar gráficamente la distribución de los 12 nodos, los servicios desplegados y la interacción entre ellos. Incluir también el esquema conceptual de federación.

Comparaciones técnicas

Elaborar una tabla comparativa entre Docker Swarm y Kubernetes, destacando aspectos como escalabilidad, facilidad de configuración, tolerancia a fallos y soporte para arquitecturas complejas.

Análisis final y conclusiones

Redactar un análisis integrador que evalúe las ventajas y desventajas de cada enfoque de orquestación. Relacionar los resultados con escenarios reales, tales como infraestructuras empresariales, servicios distribuidos y aplicaciones críticas. Proponer casos de uso donde la federación de clústeres resulte más adecuada.